

LOST & FOUND PORTAL

Management System — SPM Documentation

Software Project Management (SPM) — Group Project

Course	Software Project Management (SPM)
Submitted To	Mam Amna Shaukat
■ ■ Member 1 ■ ■	
Name	M. Mughees Mahdi
Registration No.	70146406
■ ■ Member 2 ■ ■	
Name	Saad latif
Registration No.	70146591
GitHub Link	github.com/70146406-maker/LOST-AND-FOUND-MANAGEMENT-SYSTEM

1. Project Scenario Description

1.1 Overview

The **Lost & Found Portal Management System** is a web-based application designed to help people report lost items and locate misplaced belongings efficiently. Users submit details and images of lost or found items, and the system intelligently matches related posts to reunite owners with their belongings.

The platform is especially practical for schools, universities, offices, and other public institutions where items are frequently misplaced. It digitizes the traditional lost-and-found process, making it faster, more organized, and accessible to all users.

1.2 Problem description

Universities are large, densely populated environments where students, faculty, and staff constantly move between classrooms, libraries, cafeterias, hostels, and sports facilities. As a natural consequence, personal belongings such as ID cards, wallets, mobile phones, keys, textbooks, laptops, and bags are frequently lost or misplaced on campus.

Currently, most universities rely on informal, unorganized methods to handle lost and found items - such as posting on WhatsApp groups, verbal announcements, physical notice boards, or dropping items off at a single, often untracked, security desk. These methods suffer from several shortcomings:

Lack of a centralized record, making it difficult to track what has been lost or found.

No searchable database - students must manually check multiple groups or boards.

Delayed communication between the person who lost an item and the person who found it.

No verification process, increasing the risk of false claims on found items.

Items often remain unclaimed indefinitely simply because the right people never see the post.

This results in significant inconvenience, financial loss, and frustration for students and staff, and reflects poorly on the university's administrative efficiency. There is a clear need for a dedicated, centralized, and easy-to-use Lost and Found Portal that streamlines the process of reporting, searching, and reclaiming lost items within the university community.

2. Methodology

This project will follow the Agile (Scrum) methodology, which allows the development process to be broken down into manageable iterations (sprints), enabling continuous feedback, testing, and refinement throughout the project lifecycle. Agile is well suited to this project because requirements (such as UI preferences and matching logic) may evolve as user feedback is gathered.

2.1 Development Approach

1. Requirement Gathering - Conduct surveys/interviews with students to understand common pain points in the current lost-and-found process.
2. System Design - Create ER diagrams, use-case diagrams, and wireframes for the web/mobile interface.
3. Sprint-based Development - Build the system incrementally in three sprints (authentication & posting, search & matching, claim & verification).
4. Testing - Perform unit, integration, and user acceptance testing (UAT) with a sample group of students.
5. Deployment - Host the application on a cloud platform (e.g., Firebase, Heroku, or a university server) for live use.

2.2 Tools and Technologies

- Front-end: React.js / Flutter (for web and mobile interface)
- Back-end: Node.js with Express.js
- Database: MongoDB / MySQL
- Authentication: University email-based login / Firebase Auth
- Version Control: Git & GitHub
- Project Management: Trello / Jira for sprint tracking

2.3 Project Timeline

Phase	Activities	Duration
1. Requirement Gathering & Analysis	Identify stakeholders, gather functional & non-functional requirements, define scope	Week 1-2
2. System Design	Design database schema, UI/UX wireframes, system architecture (ERD, DFD, use-case diagrams)	Week 2-3
3. Development - Sprint 1	Build user authentication, item posting module (lost/found forms)	Week 3-4
4. Development - Sprint 2	Build search/filter functionality, matching & notification system	Week 5-6
5. Development - Sprint 3	Build claim request & admin verification module	Week 7
6. Testing	Unit testing, integration testing, user acceptance testing (UAT) with sample students	Week 8
7. Deployment & Documentation	Deploy to hosting platform, finalize project report and presentation	Week 9-10

3. W1H Principle

The 5W1H framework is used to clearly define the scope, purpose, and execution plan of the project by answering six fundamental questions:

Element	Description
WHAT	A web/mobile-based Lost and Found Portal for the university that allows students, faculty, and staff to report lost items, post found items, search listings, and coordinate the return of belongings through a centralized, verified platform.
WHY	Currently, lost items on campus are reported informally (WhatsApp groups, notice boards, word of mouth), which is unorganized, slow, and often results in items never being returned to their rightful owners. A centralized portal solves this by providing a structured, searchable, and trustworthy system.
WHO	Primary users: students, faculty, and administrative staff. Secondary stakeholders: university security office / student affairs department (who may moderate claims), and the development team (project group) responsible for building and maintaining the system.
WHEN	The project will be developed and delivered within the current semester timeline, following the phased schedule defined in the Methodology section (typically 6-10 weeks depending on course requirements).

WHERE	The portal will be used within the university campus environment - accessible via web browser and mobile devices to all members of the university community, hosted on a cloud platform accessible from anywhere with internet access.
HOW	The system will be developed using an Agile methodology, with a front-end interface for posting/searching items, a back-end server handling authentication and matching logic, and a database storing item records, user accounts, and claim requests.

3.1 Problem Statement

Traditional lost-and-found systems rely on physical notice boards, word-of-mouth, or manual record-keeping — all slow and inefficient. There is no centralized, searchable way for people to report or claim items. This portal solves that problem by providing a structured digital platform for the entire lifecycle of a lost or found report.

3.2 Target Users

- Students and staff in universities or schools
- Visitors to public places (malls, airports, offices)
- Administrators managing institutional lost-and-found departments
- Anyone who has lost or found an item in a shared space

4. Design Overview

4.1 Design Principles

- Normalized relational structure (3NF) to eliminate data redundancy
- Proper Primary Keys (PK) and Foreign Keys (FK) for referential integrity
- ENUM types for constrained/status fields (role, status, match_status)
- TIMESTAMP fields for full audit trails (created_at, sent_at, matched_at)
- UNIQUE constraint on email to prevent duplicate accounts

4.2 Tables at a Glance

Table	Primary Key	Purpose
Users	user_id	Registered user accounts and profiles
Categories	category_id	Lookup table for item classification
Lost_Items	lost_id	Items reported as lost by users
Found_Items	found_id	Items reported as found by users
Messages	msg_id	Private messages between users
Matches	match_id	System-generated matches between lost & found items

5. Tables and Relationships

5.1 Users

Stores all registered users. Each user can report multiple lost or found items and exchange messages.

Column	Type	Constraints	Description
user_id	INT	PK, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(150)	NOT NULL, UNIQUE	Login email address
password	VARCHAR(255)	NOT NULL	Hashed password
phone	VARCHAR(20)	—	Contact number
role	ENUM('user','admin')	DEFAULT 'user'	Access level
profile_image	VARCHAR(255)	—	Path to profile picture
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time

5.2 Categories

Admin-managed lookup table for item classification. Used to filter and improve matching accuracy.

1. Column	Type	Constraints	Description
category_id	INT	PK, AUTO_INCREMENT	Unique category ID
category_name	VARCHAR(100)	NOT NULL	Category label
description	TEXT	—	What items belong in this category

5.3 Lost Items

Records items reported as lost. References Users via FK.

Column	Type	Constraints	Description
lost_id	INT	PK, AUTO_INCREMENT	Unique report ID
user_id	INT	FK → Users	Reporter
item_name	VARCHAR(150)	NOT NULL	Name of the item
category	VARCHAR(100)	—	Item category
description	TEXT	—	Detailed description
location_lost	VARCHAR(255)	—	Where it was lost
date_lost	DATE	—	Date lost
status	ENUM(...)	DEFAULT 'lost'	lost / found / closed
created_at	TIMESTAMP	DEFAULT NOW()	Report time

5.4 Found_Items

Records items reported as found. Supports image upload. References Users via FK.

Column	Type	Constraints	Description
found_id	INT	PK, AUTO_INCREMENT	Unique report ID
user_id	INT	FK → Users	Finder
item_name	VARCHAR(150)	NOT NULL	Name of the item
description	TEXT	—	Detailed description
category	VARCHAR(100)	—	Item category
location_found	VARCHAR(255)	—	Where it was found
date_found	DATE	—	Date found
image	VARCHAR(255)	—	Uploaded image path
status	ENUM(...)	DEFAULT 'available'	available / claimed / closed
created_at	TIMESTAMP	DEFAULT NOW()	Report time

5.5 Messages

Private messaging between users. Both sender and receiver are FKs referencing Users.

Column	Type	Constraints	Description
msg_id	INT	PK, AUTO_INCREMENT	Unique message ID
sender_id	INT	FK → Users	Sender
receiver_id	INT	FK → Users	Recipient
subject	VARCHAR(255)	—	Message subject
msg	TEXT	NOT NULL	Message body
sent_at	TIMESTAMP	DEFAULT NOW()	Sent time
is_read	BOOLEAN	DEFAULT FALSE	Read flag

5.6 Matches

Junction table linking Lost_Items and Found_Items. Implements the many-to-many relationship with a confidence score.

Column	Type	Constraints	Description
match_id	INT	PK, AUTO_INCREMENT	Unique match ID
lost_id	INT	FK → Lost_Items	Matched lost item
found_id	INT	FK → Found_Items	Matched found item
match_score	DECIMAL(5,2)	—	Confidence score (0–100)
match_status	ENUM(...)	DEFAULT 'pending'	pending / confirmed
matched_at	TIMESTAMP	DEFAULT NOW()	Match creation time

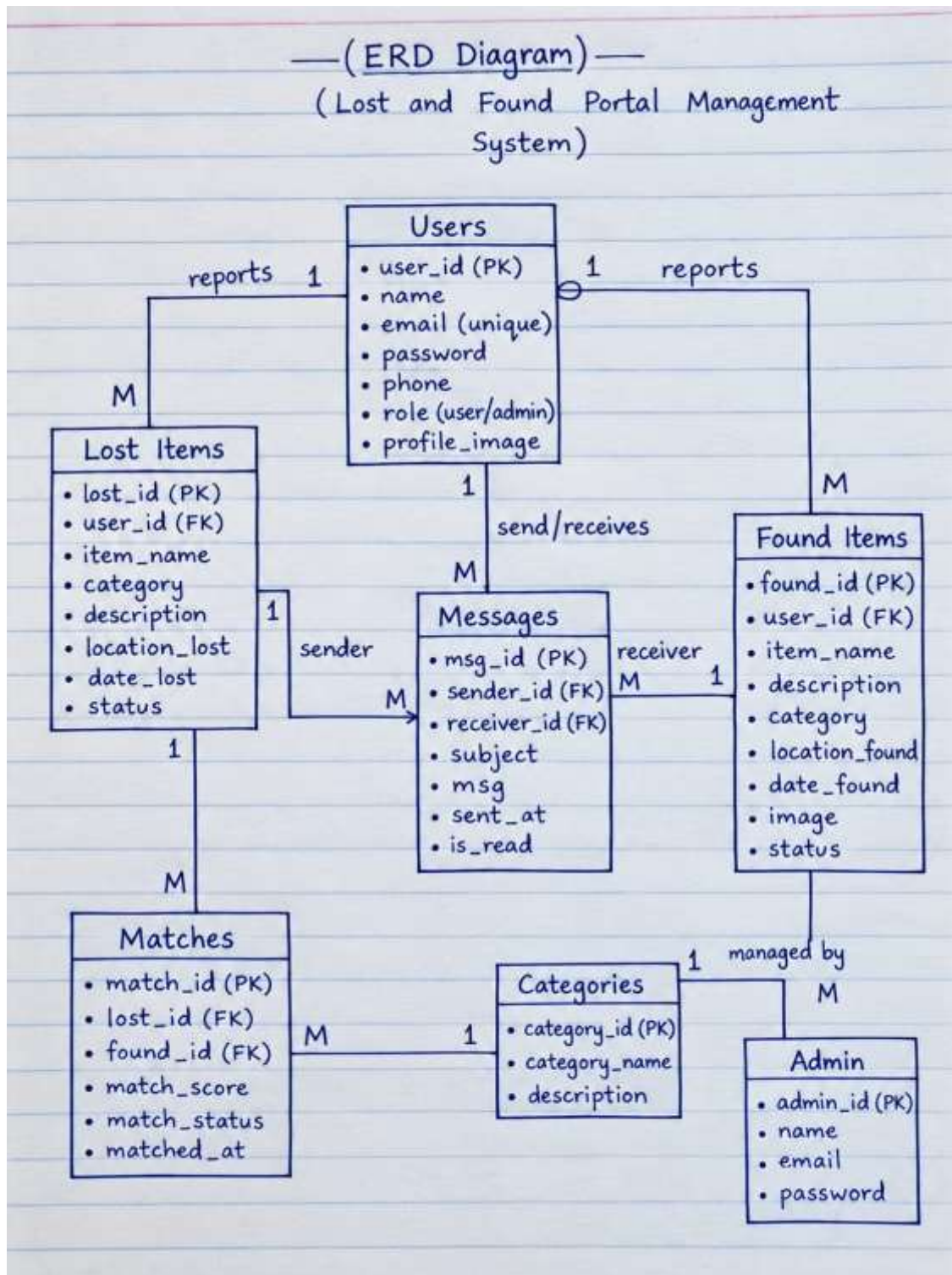
5.7 Relationship Summary

Relationship	Cardinality	Description
Users → Lost_Items	1 : M	One user can report many lost items
Users → Found_Items	1 : M	One user can report many found items
Users → Messages (sender)	1 : M	One user can send many messages
Users → Messages (receiver)	1 : M	One user can receive many messages
Lost_Items ↔ Found_Items	M : M	Via Matches junction table
Found_Items → Categories	M : 1	Each item belongs to one category

6. ER Diagram

6.1 Hand-drawn Entity Relationship Diagram

The following ERD was designed by the team prior to SQL implementation. It maps all entities, their attributes, primary/foreign keys, and the cardinality of relationships between tables.



7. Description of the 10 Features

Feature 1 — User Registration & Login

Allows new users to create an account with name, email, password, and phone. Supports role-based access (user/admin) and secure login using hashed passwords.

```
-- DDL: Create Users table
CREATE TABLE Users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  phone VARCHAR(20),
  role ENUM('user','admin') DEFAULT 'user',
  profile_image VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- DML: Register a new user
INSERT INTO Users (name, email, password, phone, role)
VALUES ('Abdullah Ayoub', 'abdullah@email.com', 'hashed_pass1', '0300-1234567', 'user');
-- DCL: Grant read access to app role
GRANT SELECT ON Users TO 'app_user'@'localhost';
```

Feature 2 — Report a Lost Item

Users submit a report specifying item name, category, description, location, and date. The record is saved with a default status of 'lost'.

```
-- DML: Insert a lost item report
INSERT INTO Lost_Items (user_id, item_name, category, description, location_lost, date_lost)
VALUES (1, 'iPhone 14', 'Electronics', 'Black iPhone 14 with cracked screen', 'Canteen Area', '2026-06-01');
-- DML: View all active lost reports
SELECT * FROM Lost_Items WHERE status = 'lost' ORDER BY date_lost DESC;
```

Feature 3 — Report a Found Item

Users report an item they found with description, location, date, and an optional image. The item is saved with 'available' status for others to claim.

```
-- DML: Insert a found item report
INSERT INTO Found_Items (user_id, item_name, description, category, location_found, date_found)
VALUES (3, 'Black iPhone', 'Black phone found near canteen', 'Electronics', 'Canteen', '2026-06-01');
-- DML: View all available found items
SELECT * FROM Found_Items WHERE status = 'available' ORDER BY date_found DESC;
```

Feature 4 — Search & Filter Items

Users can search by item name and filter results by category, location, or date range, making it easy to find relevant reports quickly.

```
-- DML: Search by item name
SELECT * FROM Lost_Items WHERE item_name LIKE '%iPhone%';
-- DML: Filter found items by category and location
SELECT * FROM Found_Items
WHERE category = 'Electronics' AND location_found = 'Canteen';
-- DML: Filter lost items by date range
SELECT * FROM Lost_Items
WHERE date_lost BETWEEN '2026-06-01' AND '2026-06-05';
```

Feature 5 — Matching System

The system suggests matches between lost and found items. Each match is assigned a confidence score (0–100) and a status of 'pending' or 'confirmed'. Uses multi-table JOIN.

```
-- DDL: Create Matches table
CREATE TABLE Matches (
  match_id INT AUTO_INCREMENT PRIMARY KEY,
  lost_id INT NOT NULL,
  found_id INT NOT NULL,
  match_score DECIMAL(5,2),
  match_status ENUM('pending','confirmed') DEFAULT 'pending',
  matched_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (lost_id) REFERENCES Lost_Items(lost_id),
  FOREIGN KEY (found_id) REFERENCES Found_Items(found_id)
);

-- DML: View matches with item and user names (JOIN)
SELECT M.match_id, L.item_name AS lost_item, F.item_name AS found_item,
  U1.name AS lost_by, U2.name AS found_by,
  M.match_score, M.match_status
FROM Matches M
JOIN Lost_Items L ON M.lost_id = L.lost_id
JOIN Found_Items F ON M.found_id = F.found_id
JOIN Users U1 ON L.user_id = U1.user_id
JOIN Users U2 ON F.user_id = U2.user_id;
```

Feature 6 — Messaging / Contact Feature

Users send private messages to each other to discuss a lost or found item. Each message has a subject, body, timestamp, and a read/unread flag.

```
-- DDL: Create Messages table
CREATE TABLE Messages (
  msg_id INT AUTO_INCREMENT PRIMARY KEY,
  sender_id INT NOT NULL,
  receiver_id INT NOT NULL,
  subject VARCHAR(255),
  msg TEXT NOT NULL,
  sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  is_read BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (sender_id) REFERENCES Users(user_id),
  FOREIGN KEY (receiver_id) REFERENCES Users(user_id)
);
-- DML: Send a message
INSERT INTO Messages (sender_id, receiver_id, subject, msg)
VALUES (1, 3, 'About Lost iPhone', 'Hi, I think you found my phone.');
```

```
-- DML: View inbox for a user
SELECT * FROM Messages WHERE receiver_id = 3 ORDER BY sent_at DESC;
```

Feature 7 — Update Item Status

Users and admins can update the status of lost items (lost → closed) and found items (available → claimed) once the matter is resolved. Confirms matches as well.

```
-- DML: Mark a lost item as closed
UPDATE Lost_Items SET status = 'closed' WHERE lost_id = 1;
-- DML: Mark a found item as claimed
UPDATE Found_Items SET status = 'claimed' WHERE found_id = 1;
-- DML: Confirm a system-suggested match
UPDATE Matches SET match_status = 'confirmed' WHERE match_id = 1;
```

Feature 8 — Delete / Remove Reports

Users can delete their own reports; admins can remove any report. Supports bulk cleanup of closed records to keep the database lean.

```
-- DML: User deletes their own lost item report
DELETE FROM Lost_Items WHERE lost_id = 5 AND user_id = 3;
-- DML: Admin removes a specific found item
DELETE FROM Found_Items WHERE found_id = 3;
-- DML: Bulk cleanup of all closed records
DELETE FROM Lost_Items WHERE status = 'closed';
DELETE FROM Found_Items WHERE status = 'closed';
```

Feature 9 — Category Management (Admin)

Admins can add, update, or delete item categories. Categories improve search accuracy and matching quality across the platform.

```
-- DDL: Create Categories table
CREATE TABLE Categories (
  category_id INT AUTO INCREMENT PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL,
  description TEXT
);
-- DML: Add a new category
INSERT INTO Categories (category_name, description)
VALUES ('Electronics', 'Mobile phones, laptops, chargers, etc.');
```

```
-- DML: Update a category description
UPDATE Categories
SET description = 'All electronic devices and accessories'
WHERE category_name = 'Electronics';
-- DML: Remove an unused category
DELETE FROM Categories WHERE category_id = 7;
```

Feature 10 — User Activity Report

Generates a summary showing each user's activity: total lost reports, found reports, and messages sent. Uses aggregate functions with GROUP BY and LEFT JOIN.

```
-- DML: User activity summary
SELECT
  U.name,
  COUNT(DISTINCT L.lost_id) AS total_lost,
  COUNT(DISTINCT F.found_id) AS total_found,
  COUNT(DISTINCT MSG.msg_id) AS messages_sent
FROM Users U
LEFT JOIN Lost_Items L ON U.user_id = L.user_id
LEFT JOIN Found_Items F ON U.user_id = F.user_id
LEFT JOIN Messages MSG ON U.user_id = MSG.sender_id
GROUP BY U.user_id, U.name
ORDER BY total_lost DESC;
```

6. SQL Execution — Screenshots

The following screenshots were captured from OneCompiler (MySQL) demonstrating actual execution of the SQL scripts. All DDL, DML, and DCL commands were successfully run and the output verified.

Screenshot 1

CREATE TABLE for Users, Categories, Lost_Items — output shows lost item records, multi-table JOIN match results, and the user activity summary report.

The screenshot shows the OneCompiler MySQL interface with the following SQL commands and their outputs:

```

1 CREATE TABLE Users {
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   name VARCHAR(100) NOT NULL,
4   email VARCHAR(100) NOT NULL UNIQUE,
5   password VARCHAR(255) NOT NULL,
6   phone VARCHAR(20),
7   role ENUM('user', 'admin') DEFAULT 'user',
8   profile_image VARCHAR(255),
9   created_at TIMESTAMP DEFAULT CURRENT_TIME
10 };
11
12 CREATE TABLE Categories {
13   category_id INT AUTO_INCREMENT PRIMARY KEY,
14   category_name VARCHAR(100) NOT NULL,
15   description TEXT
16 };
17
18 CREATE TABLE Lost_Items {
19   lost_id INT AUTO_INCREMENT PRIMARY KEY,
20   user_id INT NOT NULL,
21   item_name VARCHAR(100) NOT NULL,
22   category VARCHAR(100),
23   description TEXT,
24   location_lost VARCHAR(255),
25   date_lost DATE,
26   status ENUM('lost', 'found', 'closed') DE
27   created_at TIMESTAMP DEFAULT CURRENT_TIME
28   FOREIGN KEY (user_id) REFERENCES Users(u
29 };
30
31 CREATE TABLE Found_Items {
32   found_id INT AUTO_INCREMENT PRIMARY KEY,
33   user_id INT NOT NULL,
34   item_name VARCHAR(100) NOT NULL,
35   category VARCHAR(100),
36   description TEXT,
37   location_found VARCHAR(255),
38   date_found DATE,
39   image VARCHAR(255),
40   status ENUM('available', 'claimed', 'clim
41   created_at TIMESTAMP DEFAULT CURRENT_TIME
42   FOREIGN KEY (user_id) REFERENCES Users(u
43 };
44
45 CREATE TABLE Messages {
46   msg_id INT AUTO_INCREMENT PRIMARY KEY,
47   sender_id INT NOT NULL,
48   receiver_id INT NOT NULL,
49   subject VARCHAR(255),
50   msg TEXT NOT NULL,
51   sent_at TIMESTAMP DEFAULT CURRENT_TIME
52   is_read BOOLEAN DEFAULT FALSE,
53   FOREIGN KEY (sender_id) REFERENCES Users(
54   FOREIGN KEY (receiver_id) REFERENCES User
55 };
56
57 CREATE TABLE Matches {
58   match_id INT AUTO_INCREMENT PRIMARY KEY,
59   lost_id INT NOT NULL,
60   found_id INT NOT NULL,
61   match_score DECIMAL(5,2),
62   match_status ENUM('pending', 'confirmed')
63   matched_at TIMESTAMP DEFAULT CURRENT_TIME
64   FOREIGN KEY (lost_id) REFERENCES Lost_It
65   FOREIGN KEY (found_id) REFERENCES Found_I
66 };

```

The output shows the following tables and their data:

lost_id	user_id	Item_name	category	Description	Location_lost	date_lost	status	created_at
1	1	iPhone 14	Electronics	Black iPhone 14 with cracked screen	Cartoon Area	2025-06-01	lost	2025-06-01 09:35:02
2	1	iPhone 14	Electronics	Black iPhone 14 with cracked screen	Cartoon Area	2025-06-01	lost	2025-06-01 09:35:02

match_id	lost_item	found_item	lost_by	found_by	match_score	match_status
1	1	1	Abdullah Ayoub	Sara Khan	95.00	confirmed
2	2	2	Hughes Mahdi	All Raza	98.00	pending
3	3	3	Sara Khan	Fatima Noor	98.00	confirmed
4	4	4	All Raza	Abdullah Ayoub	87.50	pending
5	5	5	Fatima Noor	Hughes Mahdi	92.00	confirmed

name	total_lost	total_found	messages_sent
Abdullah Ayoub	1	1	2
Hughes Mahdi	1	1	1
Sara Khan	1	1	1
All Raza	1	1	1
Fatima Noor	1	1	1

Screenshot 2

CREATE TABLE for Found_Items and Messages — output shows the full Users table with 6 registered accounts and the Categories reference data.

The screenshot shows the OneCompiler MySQL interface with the following SQL commands and their outputs:

```

1 CREATE TABLE Found_Items {
2   description TEXT,
3   category VARCHAR(100),
4   location_found VARCHAR(255),
5   date_found DATE,
6   image VARCHAR(255),
7   status ENUM('available', 'claimed', 'clim
8   created_at TIMESTAMP DEFAULT CURRENT_TIME
9   FOREIGN KEY (user_id) REFERENCES Users(u
10 };
11
12 CREATE TABLE Messages {
13   msg_id INT AUTO_INCREMENT PRIMARY KEY,
14   sender_id INT NOT NULL,
15   receiver_id INT NOT NULL,
16   subject VARCHAR(255),
17   msg TEXT NOT NULL,
18   sent_at TIMESTAMP DEFAULT CURRENT_TIME
19   is_read BOOLEAN DEFAULT FALSE,
20   FOREIGN KEY (sender_id) REFERENCES Users(
21   FOREIGN KEY (receiver_id) REFERENCES User
22 };
23
24 CREATE TABLE Categories {
25   category_id INT AUTO_INCREMENT PRIMARY KEY,
26   category_name VARCHAR(100) NOT NULL,
27   description TEXT
28 };
29
30 CREATE TABLE Matches {
31   match_id INT AUTO_INCREMENT PRIMARY KEY,
32   lost_id INT NOT NULL,
33   found_id INT NOT NULL,
34   match_score DECIMAL(5,2),
35   match_status ENUM('pending', 'confirmed')
36   matched_at TIMESTAMP DEFAULT CURRENT_TIME
37   FOREIGN KEY (lost_id) REFERENCES Lost_It
38   FOREIGN KEY (found_id) REFERENCES Found_I
39 };
40
41 CREATE TABLE Users {
42   user_id INT AUTO_INCREMENT PRIMARY KEY,
43   name VARCHAR(100) NOT NULL,
44   email VARCHAR(100) NOT NULL UNIQUE,
45   password VARCHAR(255) NOT NULL,
46   phone VARCHAR(20),
47   role ENUM('user', 'admin') DEFAULT 'user',
48   profile_image VARCHAR(255),
49   created_at TIMESTAMP DEFAULT CURRENT_TIME
50 };
51
52 CREATE TABLE Lost_Items {
53   lost_id INT AUTO_INCREMENT PRIMARY KEY,
54   user_id INT NOT NULL,
55   item_name VARCHAR(100) NOT NULL,
56   category VARCHAR(100),
57   description TEXT,
58   location_lost VARCHAR(255),
59   date_lost DATE,
60   status ENUM('lost', 'found', 'closed') DE
61   created_at TIMESTAMP DEFAULT CURRENT_TIME
62   FOREIGN KEY (user_id) REFERENCES Users(u
63 };
64
65 CREATE TABLE Found_Items {
66   found_id INT AUTO_INCREMENT PRIMARY KEY,
67   user_id INT NOT NULL,
68   item_name VARCHAR(100) NOT NULL,
69   category VARCHAR(100),
70   description TEXT,
71   location_found VARCHAR(255),
72   date_found DATE,
73   image VARCHAR(255),
74   status ENUM('available', 'claimed', 'clim
75   created_at TIMESTAMP DEFAULT CURRENT_TIME
76   FOREIGN KEY (user_id) REFERENCES Users(u
77 };
78
79 CREATE TABLE Messages {
80   msg_id INT AUTO_INCREMENT PRIMARY KEY,
81   sender_id INT NOT NULL,
82   receiver_id INT NOT NULL,
83   subject VARCHAR(255),
84   msg TEXT NOT NULL,
85   sent_at TIMESTAMP DEFAULT CURRENT_TIME
86   is_read BOOLEAN DEFAULT FALSE,
87   FOREIGN KEY (sender_id) REFERENCES Users(
88   FOREIGN KEY (receiver_id) REFERENCES User
89 };
90
91 CREATE TABLE Matches {
92   match_id INT AUTO_INCREMENT PRIMARY KEY,
93   lost_id INT NOT NULL,
94   found_id INT NOT NULL,
95   match_score DECIMAL(5,2),
96   match_status ENUM('pending', 'confirmed')
97   matched_at TIMESTAMP DEFAULT CURRENT_TIME
98   FOREIGN KEY (lost_id) REFERENCES Lost_It
99   FOREIGN KEY (found_id) REFERENCES Found_I
100 };

```

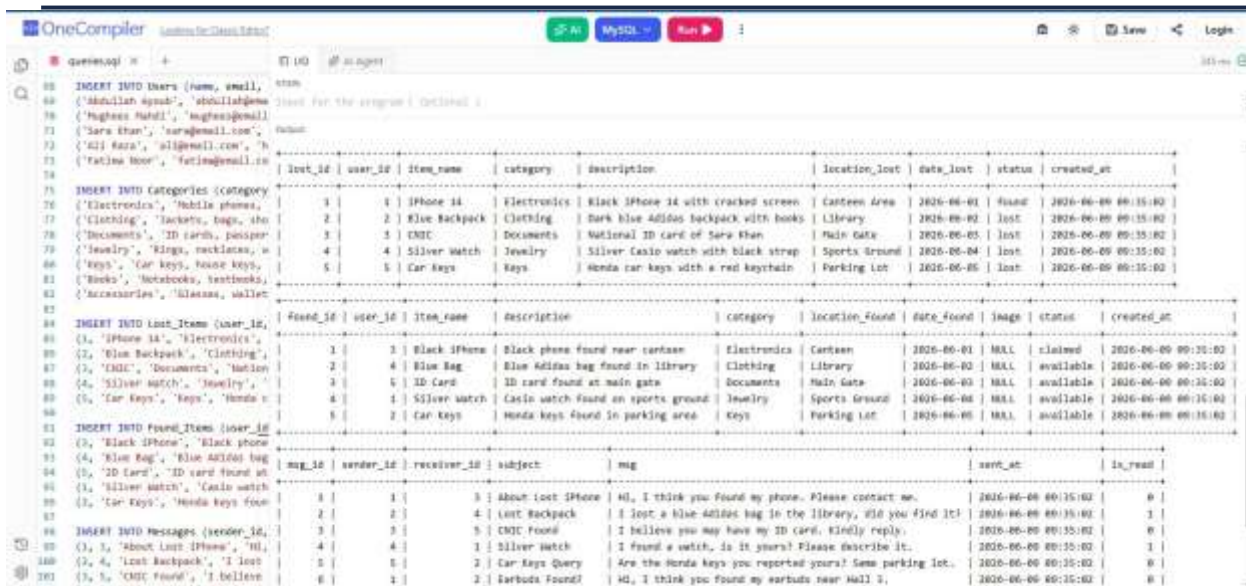
The output shows the following tables and their data:

category_id	category_name	description
1	Electronics	Mobile phones, laptops, chargers, etc.
2	Clothing	Jackets, bags, shoes, etc.
3	Documents	ID cards, passports, certificates
4	Jewelry	Rings, necklaces, watches
5	Keys	Car keys, house keys, locker keys
6	Books	Notebooks, textbooks, diaries
7	Accessories	Glasses, wallets, belts

user_id	name	email	password	phone	role	profile_image	created_at
1	Abdullah Ayoub	abdullah@gmail.com	hashed_pass1	0300-1234567	user	NAI	2025-06-01 09:35:02
2	Hughes Mahdi	hughes@gmail.com	hashed_pass2	0301-7654321	user	NAI	2025-06-01 09:35:02
3	Sara Khan	sara@gmail.com	hashed_pass3	0302-1111111	user	NAI	2025-06-01 09:35:02
4	All Raza	allr@gmail.com	hashed_pass4	0303-2222222	user	NAI	2025-06-01 09:35:02
5	Fatima Noor	fatima@gmail.com	hashed_pass5	0304-3333333	user	NAI	2025-06-01 09:35:02
6	New Student	newstudent@gmail.com	hashed_new_pass	0305-9999999	user	NAI	2025-06-01 09:35:02

Screenshot 3

INSERT INTO statements for all six tables — output shows Lost_Items, Found_Items, and Messages data populated with sample records.



The screenshot shows a MySQL editor with several queries and their results. The queries include inserting data into Users, Categories, Lost_Items, Found_Items, Messages, and Matches tables. The results are displayed in a table format.

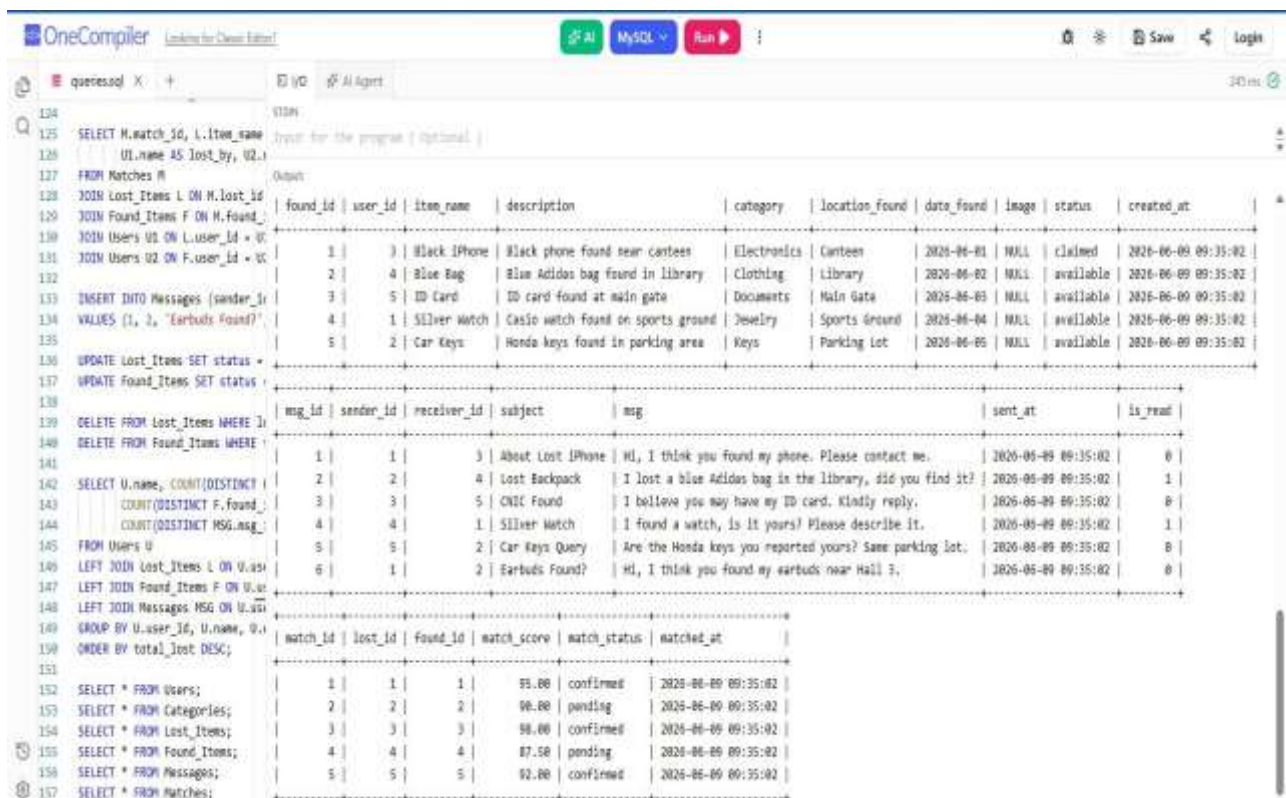
lost_id	user_id	item_name	category	Description	location_lost	date_lost	status	created_at
1	1	iPhone 14	Electronics	Black iPhone 14 with cracked screen	Canteen Area	2026-06-01	found	2026-06-09 09:35:02
2	2	Blue Backpack	Clothing	Dark blue Adidas backpack with books	Library	2026-06-02	lost	2026-06-09 09:35:02
3	3	CNIC	Documents	National ID card of Sara Khan	Main Gate	2026-06-03	lost	2026-06-09 09:35:02
4	4	Silver Watch	Jewelry	Silver Casio watch with black strap	Sports Ground	2026-06-04	lost	2026-06-09 09:35:02
5	5	Car Keys	Keys	Honda car keys with a red keychain	Parking Lot	2026-06-05	lost	2026-06-09 09:35:02

found_id	user_id	item_name	Description	category	location_found	date_found	image	status	created_at
1	3	Black iPhone	Black phone found near canteen	Electronics	Canteen	2026-06-01	NULL	claimed	2026-06-09 09:35:02
2	4	Blue Bag	Blue Adidas bag found in library	Clothing	Library	2026-06-02	NULL	available	2026-06-09 09:35:02
3	5	ID Card	ID card found at main gate	Documents	Main Gate	2026-06-03	NULL	available	2026-06-09 09:35:02
4	1	Silver Watch	Casio watch found on sports ground	Jewelry	Sports Ground	2026-06-04	NULL	available	2026-06-09 09:35:02
5	2	Car Keys	Honda keys found in parking area	Keys	Parking Lot	2026-06-05	NULL	available	2026-06-09 09:35:02

msg_id	sender_id	receiver_id	subject	msg	sent_at	is_read
1	1	3	About Lost iPhone	Hi, I think you found my phone. Please contact me.	2026-06-09 09:35:02	0
2	2	4	Lost Backpack	I lost a blue Adidas bag in the library, did you find it?	2026-06-09 09:35:02	1
3	3	5	CNIC Found	I believe you may have my ID card. Kindly reply.	2026-06-09 09:35:02	0
4	4	1	Silver Watch	I found a watch, is it yours? Please describe it.	2026-06-09 09:35:02	1
5	5	2	Car Keys Query	Are the Honda keys you reported yours? Same parking lot.	2026-06-09 09:35:02	0
6	1	2	Earbuds Found?	Hi, I think you found my earbuds near Hall 3.	2026-06-09 09:35:02	0

Screenshot 4

SELECT, UPDATE, DELETE queries — output shows Found_Items, Messages inbox, and the Matches table with scores and statuses.



The screenshot shows a MySQL editor with several queries and their results. The queries include selecting data from Matches, Found_Items, Messages, and Users tables, and updating the status of Lost_Items and Found_Items. The results are displayed in a table format.

found_id	user_id	item_name	Description	category	location_found	date_found	image	status	created_at
1	3	Black iPhone	Black phone found near canteen	Electronics	Canteen	2026-06-01	NULL	claimed	2026-06-09 09:35:02
2	4	Blue Bag	Blue Adidas bag found in library	Clothing	Library	2026-06-02	NULL	available	2026-06-09 09:35:02
3	5	ID Card	ID card found at main gate	Documents	Main Gate	2026-06-03	NULL	available	2026-06-09 09:35:02
4	1	Silver Watch	Casio watch found on sports ground	Jewelry	Sports Ground	2026-06-04	NULL	available	2026-06-09 09:35:02
5	2	Car Keys	Honda keys found in parking area	Keys	Parking Lot	2026-06-05	NULL	available	2026-06-09 09:35:02

msg_id	sender_id	receiver_id	subject	msg	sent_at	is_read
1	1	3	About Lost iPhone	Hi, I think you found my phone. Please contact me.	2026-06-09 09:35:02	0
2	2	4	Lost Backpack	I lost a blue Adidas bag in the library, did you find it?	2026-06-09 09:35:02	1
3	3	5	CNIC Found	I believe you may have my ID card. Kindly reply.	2026-06-09 09:35:02	0
4	4	1	Silver Watch	I found a watch, is it yours? Please describe it.	2026-06-09 09:35:02	1
5	5	2	Car Keys Query	Are the Honda keys you reported yours? Same parking lot.	2026-06-09 09:35:02	0
6	1	2	Earbuds Found?	Hi, I think you found my earbuds near Hall 3.	2026-06-09 09:35:02	0

match_id	lost_id	found_id	match_score	match_status	matched_at
1	1	1	95.00	confirmed	2026-06-09 09:35:02
2	2	2	90.00	pending	2026-06-09 09:35:02
3	3	3	98.00	confirmed	2026-06-09 09:35:02
4	4	4	87.50	pending	2026-06-09 09:35:02
5	5	5	92.00	confirmed	2026-06-09 09:35:02

7. GitHub Repository

7.1 Main Repository

The main repository contains the complete project including SQL scripts, schema file, and README.

URL: github.com/70146406-maker/LOST-AND-FOUND-MANAGEMENT-SYSTEM

7.2 Repository Contents

- `queries.sql` — Full schema: CREATE TABLE, INSERT, SELECT (with JOINS), UPDATE, DELETE, GRANT
- `README.md` — Project overview, feature list, and setup instructions
- `.md` feature file — Listing all 10 implemented features

8. Conclusion

The **Lost & Found Portal Management System** successfully demonstrates a complete, real-world database application built on solid relational database principles. The system implements a normalized 6-table schema with proper primary keys, foreign keys, and referential integrity across all relationships.

All 10 required features have been fully implemented covering the complete range of SQL command types: DDL (CREATE TABLE), DML (INSERT, SELECT with multi-table JOINS, UPDATE, DELETE), and DCL (GRANT). Advanced SQL techniques such as aggregate functions, GROUP BY, ENUM-based status tracking, and junction-table many-to-many relationships have been demonstrated throughout.

This project provides a practical, scalable database foundation that universities, offices, and public institutions can adopt to replace manual lost-and-found processes with a fast, organized, and fully digital system.